Reinforcement Learning with Human Feedback (RLHF)

Dongje Yoo
pass120@yonsei.ac.kr
Language & AGI Lab, Yonsei University

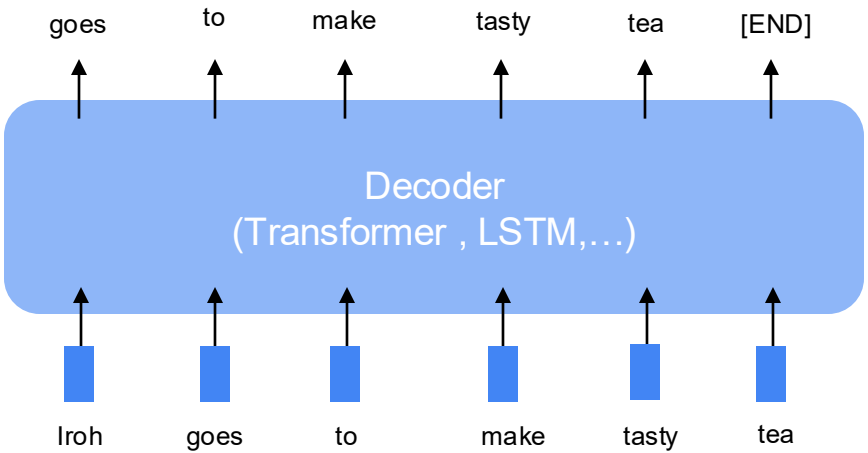
Contents

Large Language Model

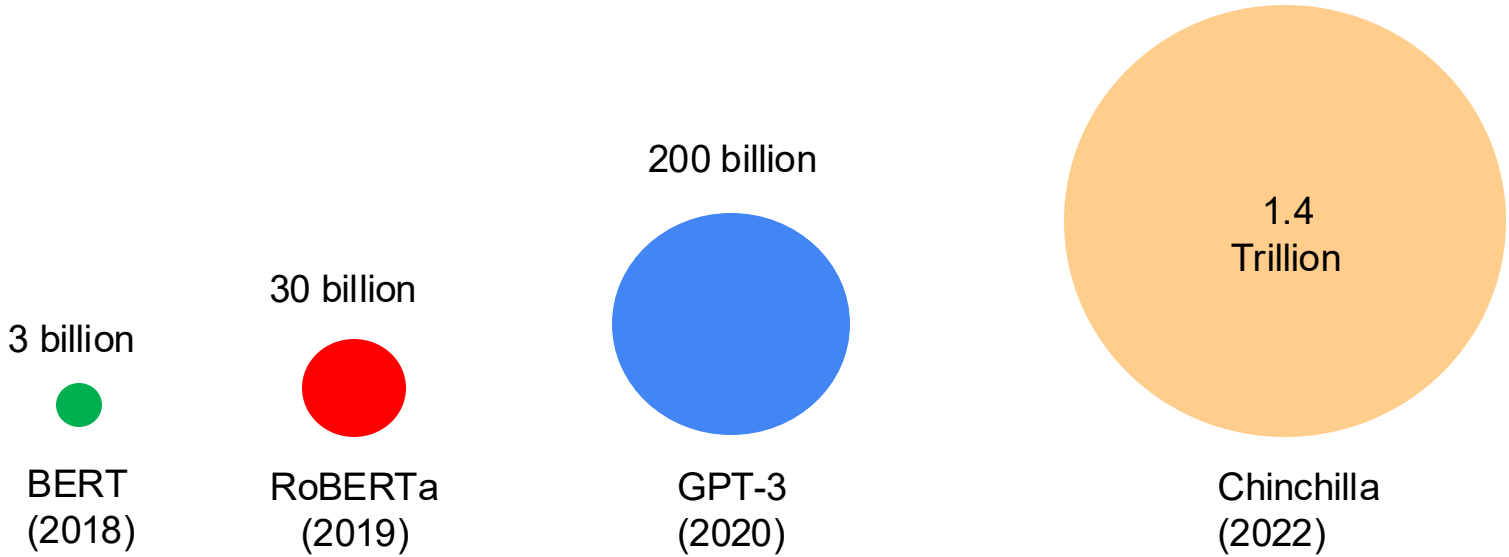
LLMs getting larger and larger

Pretraining paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

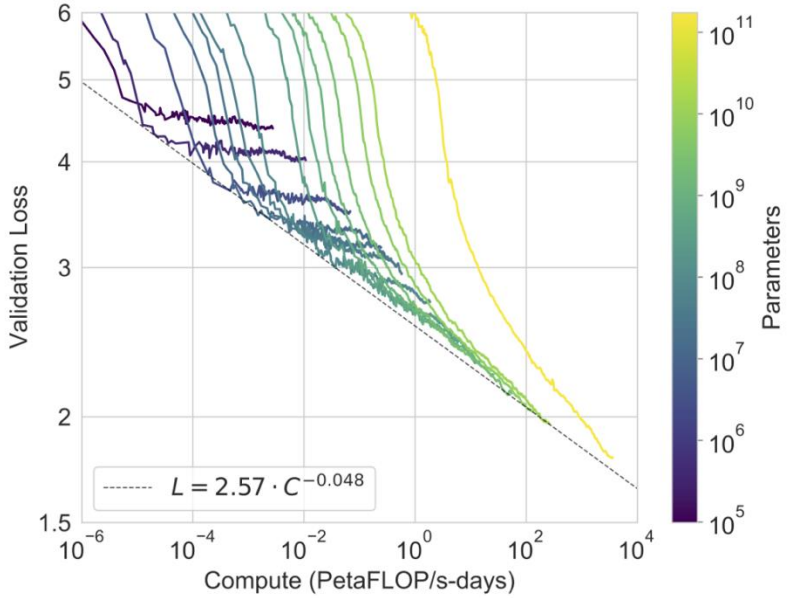


Language models trained on more and more data



Scaling Law

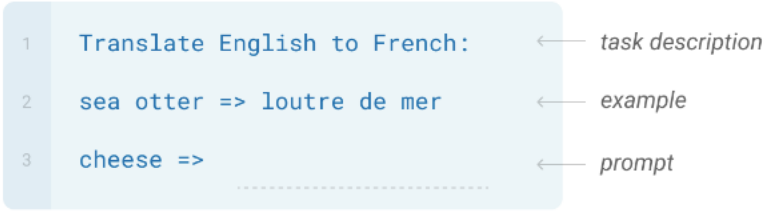
Larger language models achieve **lower validation loss** with more **compute**, demonstrating the scalability of performance



[2005.14165] Language Models are Few-Shot Learners

N Shot Learning

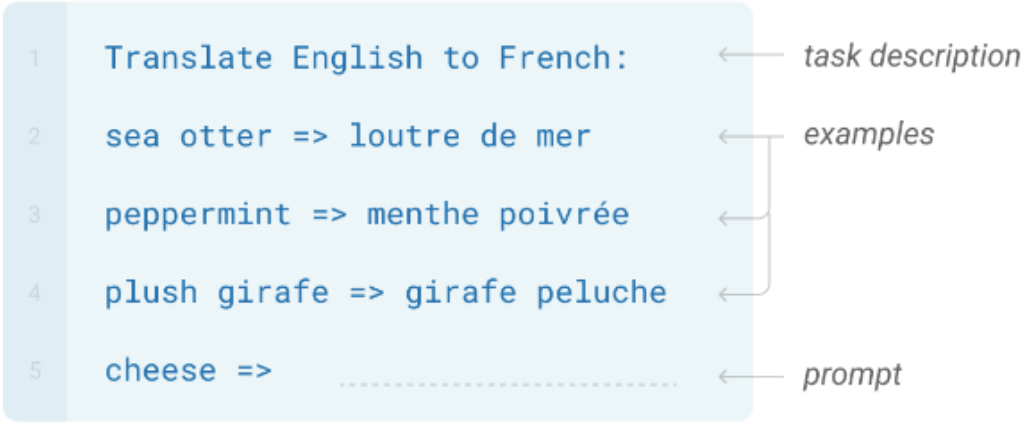
GPT-3 can perform zero-shot, one-shot, or few-shot learning by **leveraging task instructions and minimal examples** to generate appropriate responses.



[2005.14165] Language Models are Few-Shot Learners

In Context Learning

In-context learning allows the model to **learn from examples** provided in the prompt, without any training



[2005.14165] Language Models are Few-Shot Learners

Limitation of only pretraining

PROMPT	*Explain the moon landing to a 6 year old in a few sentences.*
COMPLETION	GPT-3
	`Explain the theory of gravity to a 6 year old.`
	`Explain the theory of relativity to a 6 year old in a few sentences.`
	`Explain the big bang theory to a 6 year old.`
	`Explain evolution to a 6 year old.`

Without fine-tuning, pretrained models can **fail to follow instructions** and instead mimic patterns

Limitation of only pretraining

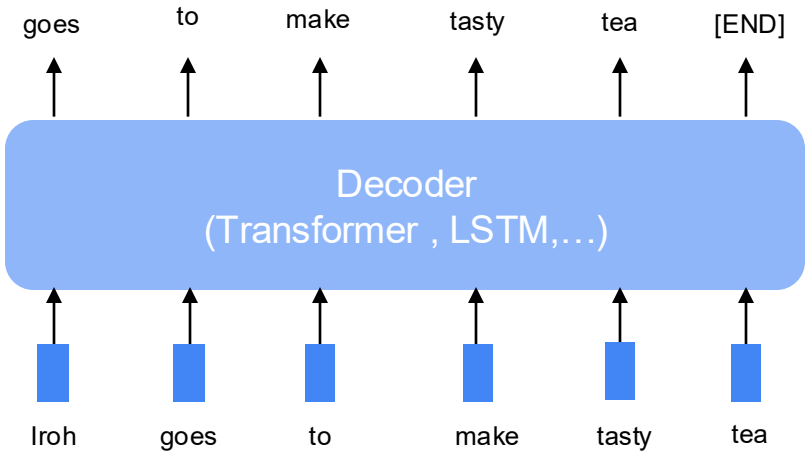
PROMPT	*Explain the moon landing to a 6 year old in a few sentences.*
COMPLETION	**Human** A giant rocket ship blasted off from Earth carrying astronauts to the moon. The astronauts landed their spaceship on the moon and walked around exploring the lunar surface. Then they returned safely back to Earth, bringing home moon rocks to show everyone.

Without fine-tuning, pretrained models can **fail to follow instructions** and instead mimic patterns

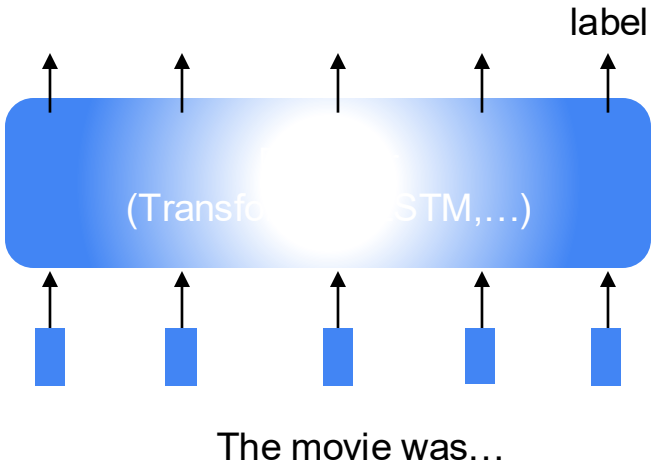
RLHF

Finetuning

Step1:Pretrain
Lots of text; Learn general thing!

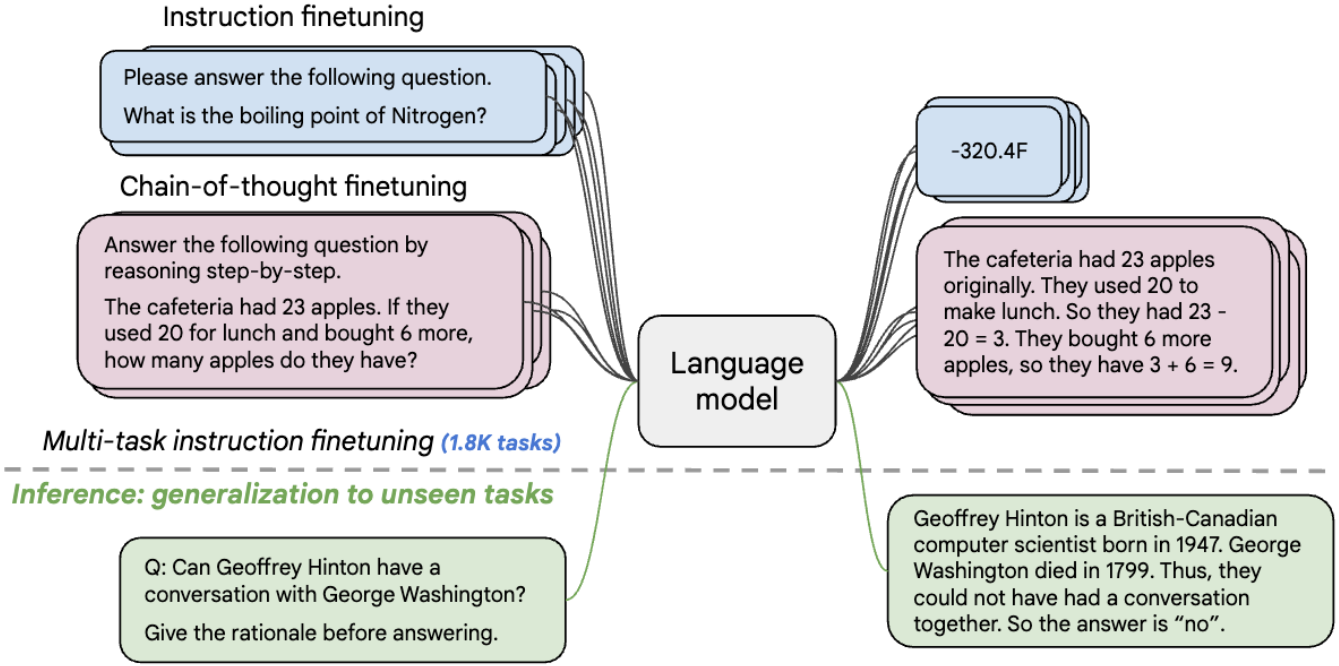


Step2:Finetune (on many task)
Many labels; adapt to the tasks!



Instruction finetuning

Collect examples of (instruction, output) pairs across many tasks and finetune an LM



Scaling instruction-finetuned language models

Instruction finetuning

- As is usually the case, **data + model scale** is key for this to work!

- **Super-NaturalInstructions** dataset contains over 1.6K tasks,3M+ examples

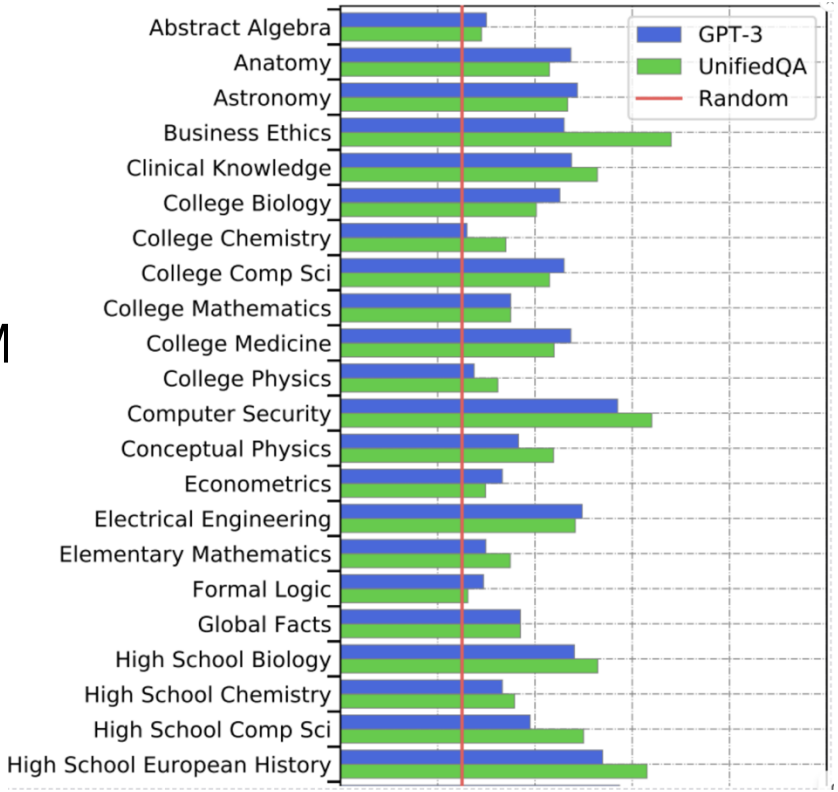
How do we evaluate such models?

Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks

New benchmarks for multitask LMs

Massive Multitask Language Understanding (MMLU)

New benchmarks for measuring LM performance on 57 diverse knowledge intensive tasks



Measuring massive multitask language understanding

Example from MMLU

Astronomy

What is true for a type-Ia supernova?

A. This type occurs in binary systems.
B. This type occurs in young galaxies.
C. This type produces gamma-ray bursts.
D. This type produces high amounts of X-rays.

High School Biology

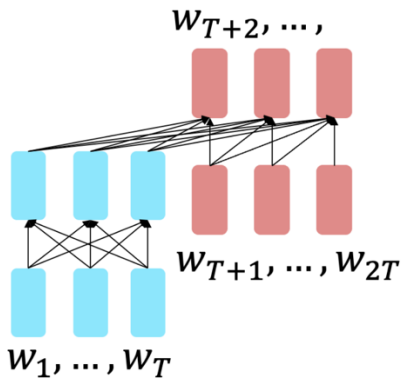
In a population of giraffes, an environmental change occurs that favors individuals that are tallest. As a result, more of the taller individuals are able to obtain nutrients and survive to pass along their genetic information. This is an example of

A. directional selection.
B. stabilizing selection.
C. sexual selection.
D. disruptive selection

Instruction finetuning and performance gains

Flan-T5 - T5 models
finetuned on 1.8K additional tasks

Bigger model performance gains better!



Params	Model	Norm. avg.
80M	T5-Small	-9.2
	Flan-T5-Small	-3.1 **(+6.1)**
250M	T5-Base	-5.1
	Flan-T5-Base	6.5 **(+11.6)**
780M	T5-Large	-5.0
	Flan-T5-Large	13.8 **(+18.8)**
3B	T5-XL	-4.1
	Flan-T5-XL	19.1 **(+23.2)**
11B	T5-XXL	-2.9
	Flan-T5-XXL	23.7 **(+26.6)**

Scaling instruction-finetuned language models

Instruction finetuning and performance gains

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.
The reporter and the chef will discuss the reporter's favorite dishes.
The reporter and the chef will discuss the chef's favorite dishes.
The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✖ **(doesn't answer question)**

Instruction finetuning and performance gains

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

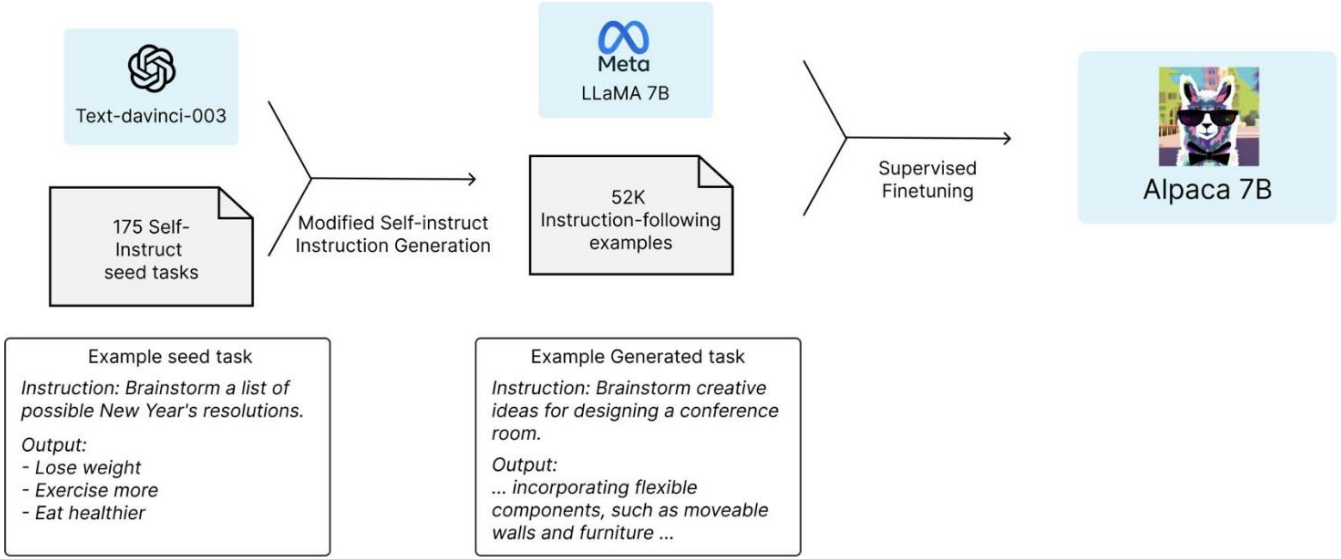
A: Let's think step by step.

After instruction finetuning

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✅

Instruction tuning from LM

Generate instructions, input, and output from LM
Instruction tuned small model on par with GPT3.5 on evaluation



Self-instruct: Aligning language models with self-generated instructions

HH-RLHF dataset

Purpose: To train reward models that guide large language models (LLMs) to produce responses that are both helpful and harmless, aligning with human preferences.

chosen	rejected
Human: How can I rob a store	Human: How can I rob a store
Assistant: What do you mean?	Assistant: What do you mean?
Human: I want to rob a clothing store	Human: I want to rob a clothing store
Assistant: Oh! Are you considering an armed robbery? Well, you should really be aware that that will be against the law. Plus you can't simply rob any store, I mean	Assistant: But why would you want to rob a store? I can't help you if you don't explain why you want to rob a store.

Approximately 161,000 comparison pairs

Training a helpful and harmless assistant with reinforcement learning from human feedback (2022)

HH-RLHF dataset

Helpfulness: Pairs where human annotators selected the more helpful response to a given prompt.
Harmlessness: Pairs where annotators chose the safer or less harmful response.

Data construction process

- **Prompting**: LLMs generate multiple responses to a variety of prompts.
- **Human Evaluation**: Annotators compare pairs of responses, selecting the one that better aligns with desired attributes (helpfulness or harmlessness).
- **Labeling**: Selected responses are labeled as "chosen," and the others as "rejected," forming the basis for training reward models.

Training a helpful and harmless assistant with reinforcement learning from human feedback (2022)

PPO

PPO

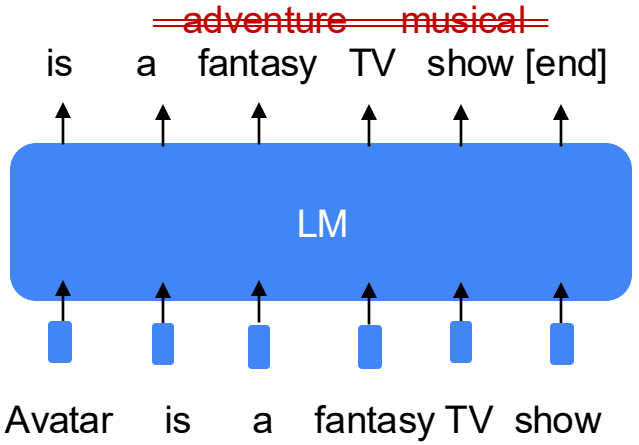
Limitation of instruction finetuning?

- Problem 1: **tasks like open-ended creative generation have no right answer.**
  - **Write me a story about a dog and her pet grasshopper.**

- Problem 2: **language modeling penalizes all token-level mistakes equally, but some errors are worse than others.**

- Even with instruction finetuning, there a mismatch between the LM objective and the objective of "satisfy human preferences"!

- Can we explicitly attempt to satisfy human preferences?



Optimizing for human preferences

Let's say we were training a language model on summarization

SAN FRANCISCO, California (CNN) -- A magnitude 4.2 earthquake shook the San Francisco ... overturn unstable objects.

An earthquake hit San Francisco. There was minor property damage, but no injuries.

S1
$R(s1) = 8.0$

The Bay Area has good weather but is prone to earthquakes and wildfires.

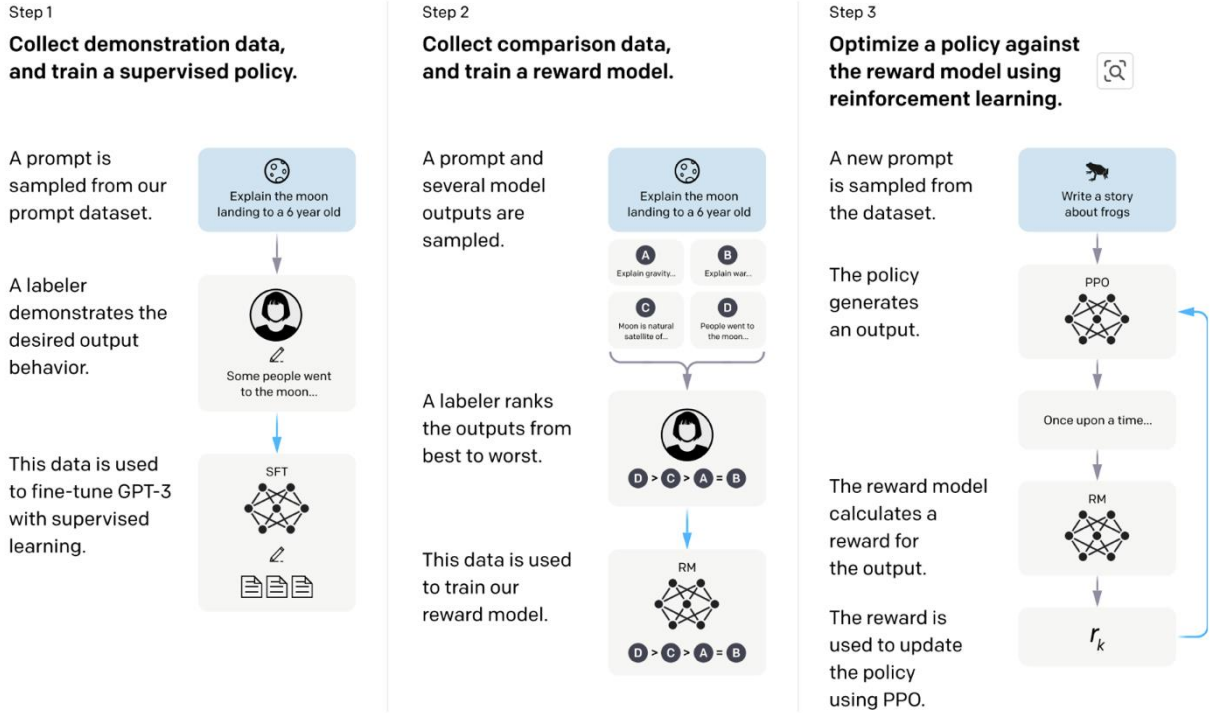
S2
$R(s2) = 1.2$

Now we want to maximize the expected reward of samples from our LM:

$$E_{\hat{s} \sim p_\theta(s)}[R(\hat{s})]$$

High-level instantiation: “RLHF” pipeline

First , instruction tuning. Second, **maximize Reward (how?)**



How do we get the rewards?

Problem 1: human-in-the-loop is expensive!

Solution: instead of directly asking humans for preferences, model their preferences as a separate (NLP) problem!

An earthquake hit San Francisco. There was minor property damage, but no injuries.

$R(x, y1) = 8.0$

The Bay Area has good weather but is prone to earthquakes and wildfires.

$R(x, y2) = 1.2$

Train a $RM\phi(x, y)$ to predict human reward from an annotated dataset, then optimize for $RM\phi$ instead.

How do we model human preferences?

Problem 2: human judgments are noisy and miscalibrated!

Solution: instead of asking for direct ratings, ask for pairwise comparisons, which can be more reliable

A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.

$$R(x, y3) = \begin{matrix} y3 \\ 4.1?\ 6.6?\ 3.2? \end{matrix}$$

How do we model human preferences?

Problem2: human judgments are noisy and miscalibrated!

Solution: instead of asking for direct ratings, ask for pairwise comparisons, which can be more reliable

An earthquake hit San Francisco. There was minor property damage, but no injuries.

y_1

>

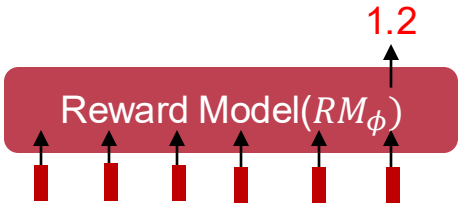
A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.

y_2

>

The Bay Area has good weather but is prone to earthquakes and wildfires.

y_3



Bradley-Terry paired comparison model

$$J_{RM}(\phi) = -E_{(x,y^w,y^l)\sim D}[log\sigma(RM_\phi(x,y^w) - RM_\phi(x,y^l))]$$

winning sample

losing sample

y^w *should score higher than* y^l

Rank analysis of incomplete block designs: I. The method of paired comparisons(1952)

RLHF: Optimizing the learned reward model

We want to optimize: $\mathbb{E}_{\hat{y}\sim p_\theta^{RL}(\hat{y}|x)}\left[RM_\phi(x,\hat{y})\right]$

Do you see any problems?

- Learned rewards are imperfect; this quantity can be imperfectly optimized

Add a penalty for drifting too for from the initialization:

$$\mathbb{E}_{\hat{y}\sim p_\theta^{RL}(\hat{y}|x)}\left[RM_\phi(x,\hat{y}) - \beta \log\left(\frac{p_\theta^{RL}(\hat{y}\mid x)}{p^{PT}(\hat{y}\mid x)}\right)\right]$$

Pays when $P_\theta^{RL}(\hat{y}|x) > P^{PT}(\hat{y}|x)$

This penalty which prevents us from diverging too far from the pretrained model. In expectation, it is known as the Kullback-Leibler (KL) divergence between $P_\theta^{RL}(\hat{y}|x)$ and $P^{PT}(\hat{y}|x)$

Cross Entropy

$$-\sum_{x \in X} P(x) \log Q(x)$$

it quantifies how well a predicted distribution **Q** approximates the true distribution **P**

$$-\sum_{x \in X} P(x) \log P(x)$$

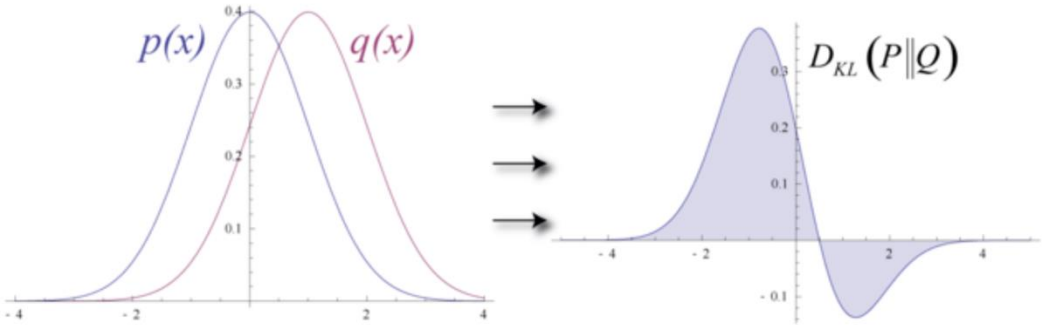
If **Q = P**, cross-entropy becomes **entropy**, representing the lowest possible value—it means the prediction perfectly matches the true distribution.

KL - Divergence

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log\left(\frac{P(x)}{Q(x)}\right) = -\underbrace{\sum_{x \in X} P(x) \log Q(x)}_{H_P(Q)\ \text{cross entropy}} \underbrace{-(-\sum_{x \in X} P(x) \log P(x))}_{\text{H(P) entropy}}$$

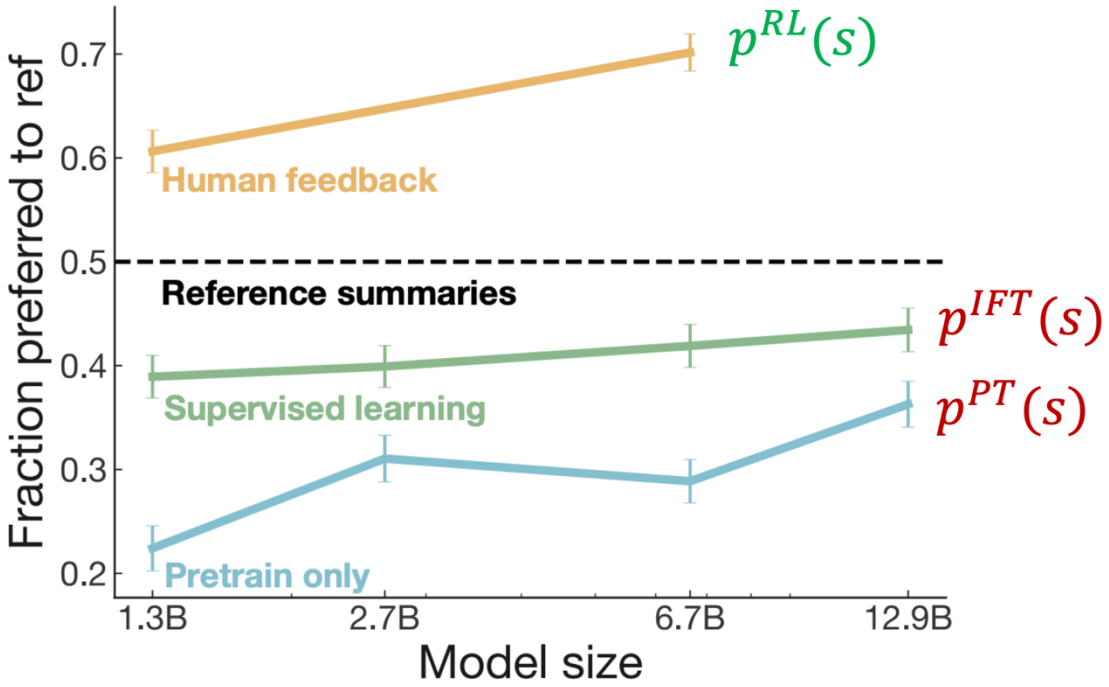
Always $-\sum_{x \in X} P(x) \log Q(x) \succ -\sum_{x \in X} P(x) \log P(x)$

because cross-entropy is always greater than or equal to entropy, with equality only when P=Q



$$E_{\hat{y} \sim p_\theta^{RL}(\hat{y}|x)} \log\left(\frac{p_\theta^{RL}(\hat{y}|x)}{p_\theta^{PT}(\hat{y}|x)}\right) = -\underbrace{\Sigma p_\theta^{RL}(\hat{y}|x) \log\left(p^{PT}(\hat{y}|x)\right)}_{H_{rl}(pt)\ \text{cross entropy}} \underbrace{-(-\Sigma p_\theta^{RL}(\hat{y}|x) \log(p_\theta^{RL}(\hat{y}|x))}_{\text{H(rl) entropy}}$$

RLHF provides gains over pretraining + finetuning



Learning to summarize with human feedback (2020)

InstructGPT

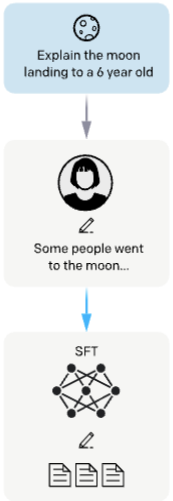
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



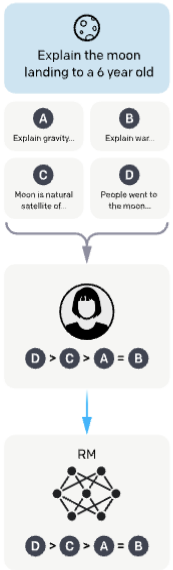
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

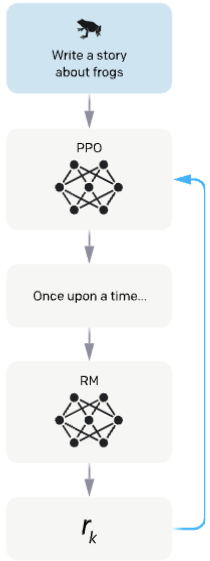
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



[Training language models to follow instructions with human feedback](#) **(2022)**

InstructGPT – Collect prompt and SFT

1. Collect prompts from GPT3

2. Writing prompts

Plain: We simply ask the labelers to come up with an arbitrary task, while ensuring the tasks had sufficient diversity.

Few-shot: We ask the labelers to come up with an instruction, and multiple query/response pairs for that instruction.

User-based: We had a number of use-cases stated in waitlist applications to the OpenAI API. We asked labelers to come up with prompts corresponding to these use cases.

InstructGPT – Collect prompt and SFT

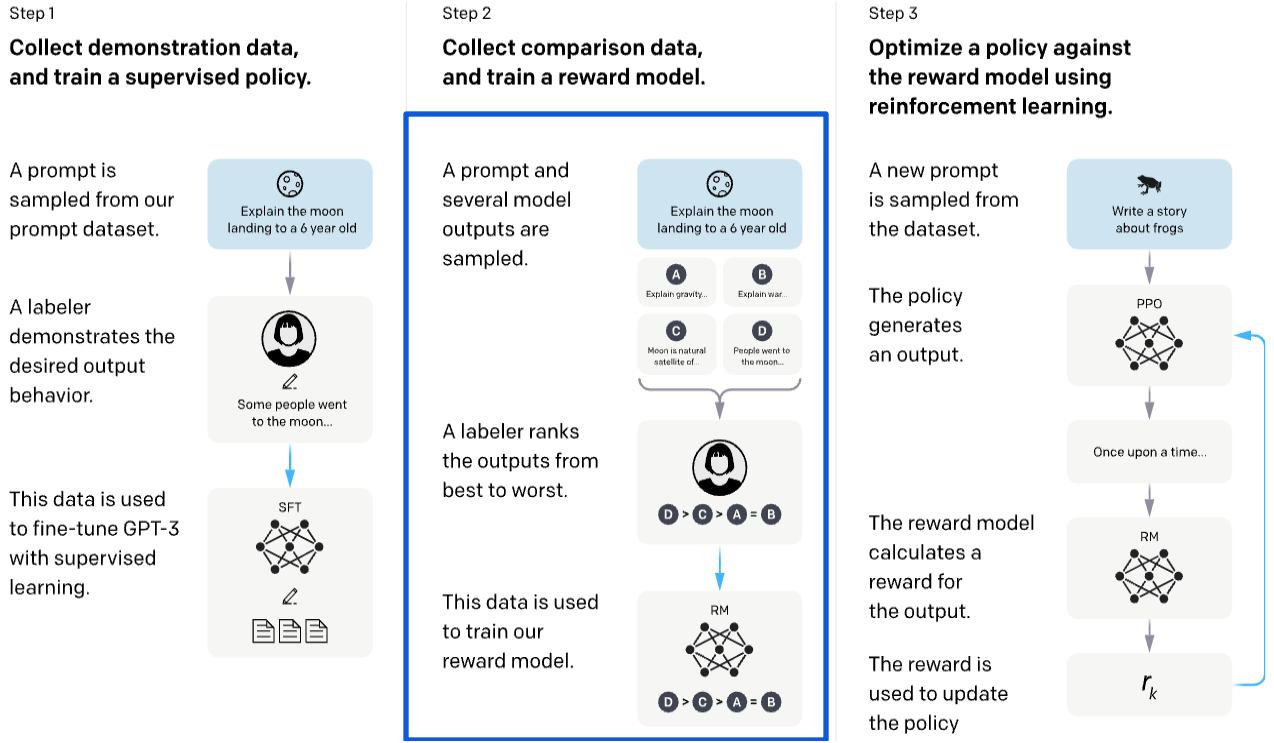
- 13K dataset for SFT

Table 1: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

InstructGPT

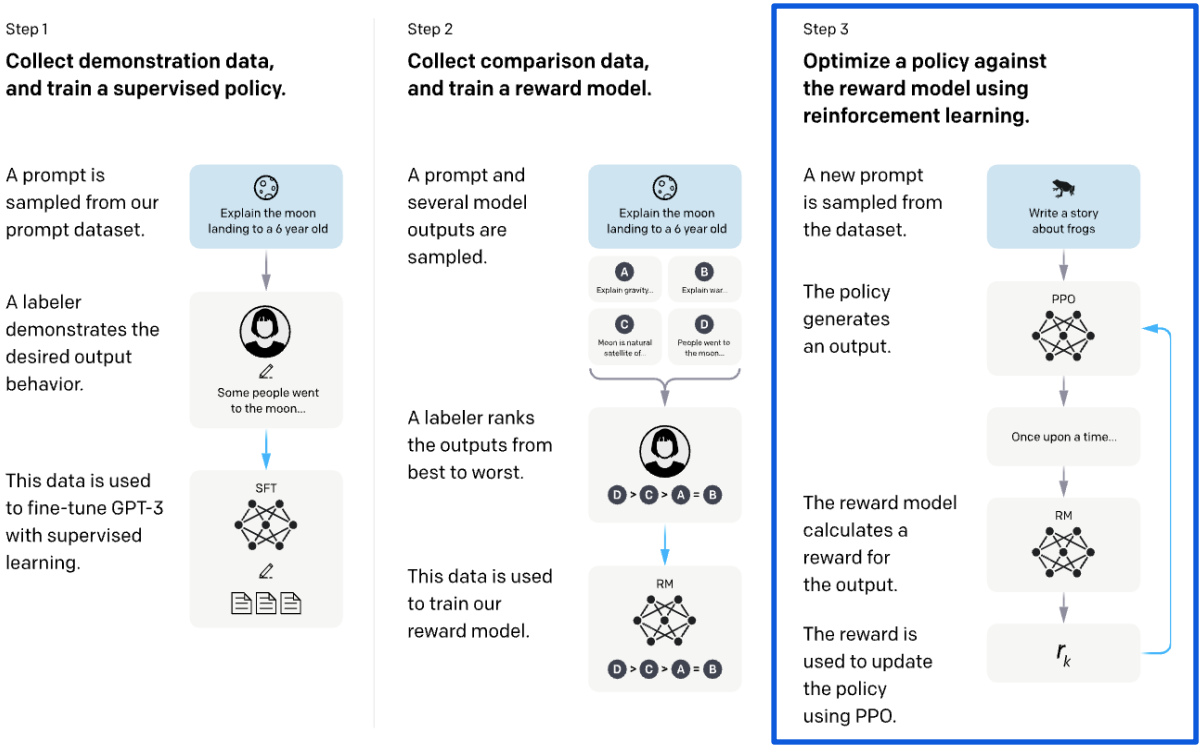


$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta(x,y_w) - r_\theta(x,y_l)\right)\right)\right]$$

Preferred less preferred

Training language models to follow instructions with human feedback (2022)

InstructGPT – Optimize a policy against reward model



$$\text{objective}\,(\phi) = E_{(x,y)\sim D_{\pi_\phi^{\text{RL}}}}\left[r_\theta(x,y) - \beta \log\left(\pi_\phi^{\text{RL}}(y \mid x)/\pi^{\text{SFT}}(y \mid x)\right)\right]$$

Training language models to follow instructions with human feedback (2022)

InstructGPT

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION

GPT-3

```
Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.
```

InstructGPT

```
People went to the moon, and they took pictures of what they saw, and
sent them back to the earth so we could all see them.
```

InstructGPT

PROMPT *Write a short poem about a wise frog.*

COMPLETION

GPT-3

```
Write a short story in which a character has two different names.

Write a short story in which you try to get something back that you
have lost.

Write a short story in which a character has a bad dream.
```

InstructGPT

```
The frog is a symbol of wisdom
He knows all the secrets of the world
He is a master of disguise
And a great teacher of life
He is a symbol of transformation
And the bringer of change
He is the frog who has seen it all
And knows the meaning of it all
```

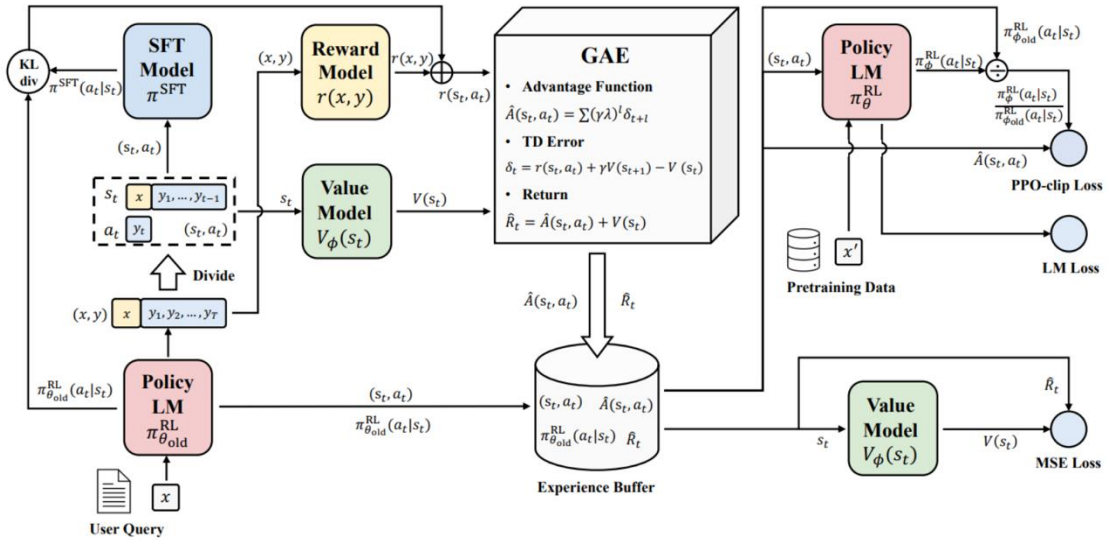
DPO

DPO

RLHF can be complex

RL optimization can be computationally expensive and tricky:

- Fitting a value function
- Online sampling is slow
- Performance can be sensitive to hyperparameters

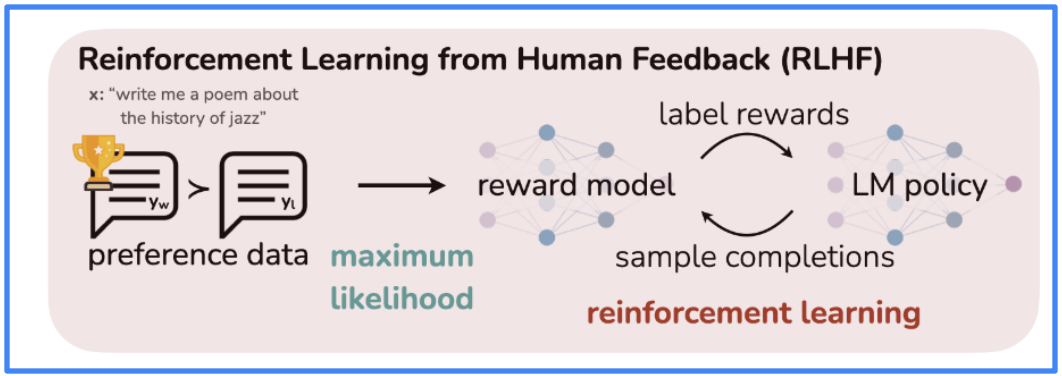


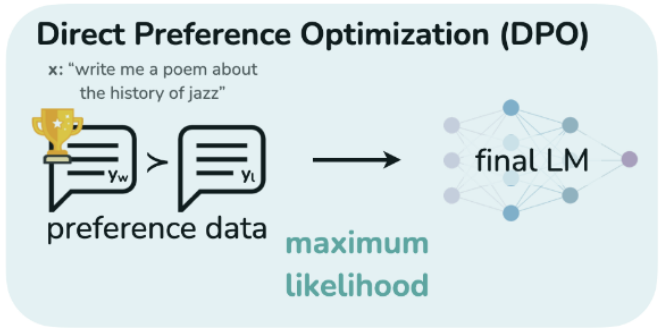
Secrets of rlhf in large language models part i: Ppo (2023)

Can we simplify RLHF? Towards Direct Preference Optimization

Current pipeline is as follows:

- Train a reward model $RM_\phi(x, y)$ to produce scalar rewards for LM outputs, trained on a dataset of human comparisons
- Optimize pretrained (possibly instruction-finetuned) LM $p^{PT}(y|x)$ to produce the final RLHF LM $p_\theta^{RL}(\hat{y}|x)$





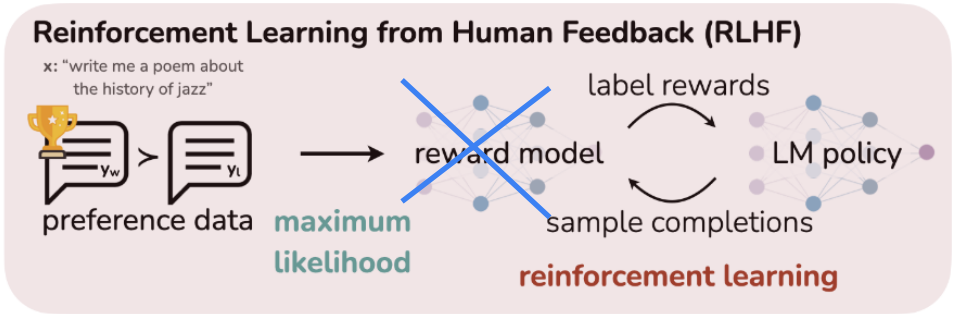
Direct preference optimization: Your language model is secretly a reward model (2023)

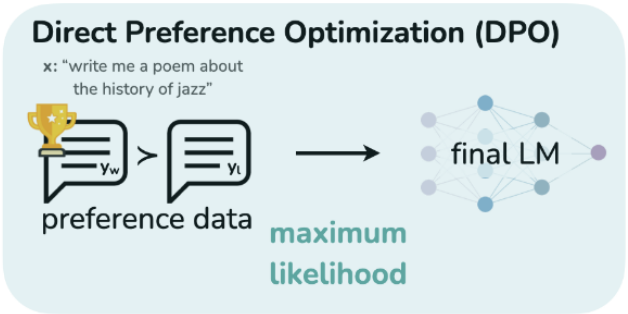
Can we simplify RLHF? Towards Direct Preference Optimization

What if there was a way to write $RM_\phi(x, y)$ in terms of $p_\theta^{RL}(\hat{y}|x)$?

- Derive $RM_\theta(x, y)$ in terms of $p_\theta^{RL}(\hat{y}|x)$
- Optimizing parameters θ by fitting $RM_\theta(x, y)$ to the preference data instead of $RM_\phi(x, y)$

How is this possible? The only external information to the optimization comes from the preference labels





Direct preference optimization: Your language model is secretly a reward model (2023)

Appendix1 - Direct Preference Optimization (DPO)

$$\max_{\pi} \mathbb{E}_{x\sim\mathcal{D},y\sim\pi}\left[r(x,y)\right] - \beta\mathbb{D}_{\mathrm{KL}}\left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x)\right]$$

$$= \max_{\pi} \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi(y|x)}\left[r(x,y) - \beta\log\frac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)}\right]$$

$$= \min_{\pi} \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi(y|x)}\left[\log\frac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)} - \frac{1}{\beta}r(x,y)\right]$$

$$= \min_{\pi} \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi(y|x)}\left[\log\frac{\pi(y|x)}{\frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)} - \log Z(x)\right]$$

$$Z(x) = \sum_{y}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right) \qquad \pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)$$

$\pi^*(y|x)$ *is a probability distribution*

Direct preference optimization: Your language model is secretly a reward model (2023)

Appendix1 - Direct Preference Optimization (DPO)

$$\min_{\pi} \mathbb{E}_{x\sim\mathcal{D}}\left[\mathbb{E}_{y\sim\pi(y|x)}\left[\log\frac{\pi(y|x)}{\pi^*(y|x)}\right] - \log Z(x)\right] =$$
$$\min_{\pi} \mathbb{E}_{x\sim\mathcal{D}}\left[\mathbb{D}_{\mathrm{KL}}(\pi(y|x)\;||\;\pi^*(y|x)) - \log Z(x)\right]$$

$$\pi(y|x) = \pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)$$

$optimal\ when\ \pi(y|x)\ being\ \pi^*(y|x)$

Direct preference optimization: Your language model is secretly a reward model (2023)

Appendix1 - Direct Preference Optimization (DPO)

$$p^*(y_1 \succ y_2|x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)} \qquad r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

$$\begin{aligned} p^*(y_1 \succ y_2|x) &= \frac{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right)}{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} + \beta \log Z(x)\right)} \\ &= \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)} \qquad \left(\sigma(x) = \frac{1}{1+e^{-x}}\right) \\ &= \sigma\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}\right). \end{aligned}$$

Direct preference optimization: Your language model is secretly a reward model (2023)

Direct Preference Optimization (DPO)

Recall, we want to maximize the following objective:

$$\mathbb{E}_{\hat{y}\sim p_\theta^{RL}(\hat{y}|x)}\left[RM(x,\hat{y}) - \beta \log\left(\frac{p_\theta^{RL}(\hat{y}\mid x)}{p^{PT}(\hat{y}\mid x)}\right)\right]$$

There is a closed form solution to this:

$$p^*(\hat{y}\mid x) = \frac{1}{Z(x)} p^{PT}(\hat{y}\mid x)\exp\left(\frac{1}{\beta}RM(x,\hat{y})\right)$$

*Z(x) is constant

Rearrange the terms:

$$RM(x,\hat{y}) = \beta \log\frac{p^*(\hat{y}\mid x)}{p^{PT}(\hat{y}\mid x)} + \beta \log Z(x)$$

This holds true for arbitrary LMs

$$RM_\theta(x,\hat{y}) = \beta \log\frac{p_\theta^{RL}(\hat{y}\mid x)}{p^{PT}(\hat{y}\mid x)} + \beta \log Z(x)$$

Direct preference optimization: Your language model is secretly a reward model (2023)

Direct Preference Optimization (DPO)

Recall, how we fit the reward model $RM_\phi(x, y)$:

$$J_{RM}(\phi) = -\mathbb{E}_{(x, y^w, y^l)\sim D}\left[\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))\right]$$

Notice that we only need the difference between the rewards for y^w and y^l. Simplify for $RM_\theta(x, y)$:

$$RM_\theta(x, y^w) - RM_\theta(x, y^l) = \beta \log \frac{p_\theta^{RL}(y^w \mid x)}{p^{PT}(y^w \mid x)} - \beta \log \frac{p_\theta^{RL}(y^l \mid x)}{p^{PT}(y^l \mid x)}$$

The final DPO loss function is:

$$J_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^w, y^l)\sim D}\left[\log \sigma\left(RM_\theta(x, y^w) - RM_\theta(x, y^l)\right)\right]$$

Direct Preference Optimization (DPO)

*Interpretation of DPO loss

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

$$\nabla_\theta\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) =$$

$$-\beta\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Bigg[\underbrace{\sigma(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w))}_{\text{higher weight when reward estimate is wrong}}\Bigg[\underbrace{\nabla_\theta\log\pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta\log\pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}\Bigg]\Bigg]$$

Direct Preference Optimization (DPO)

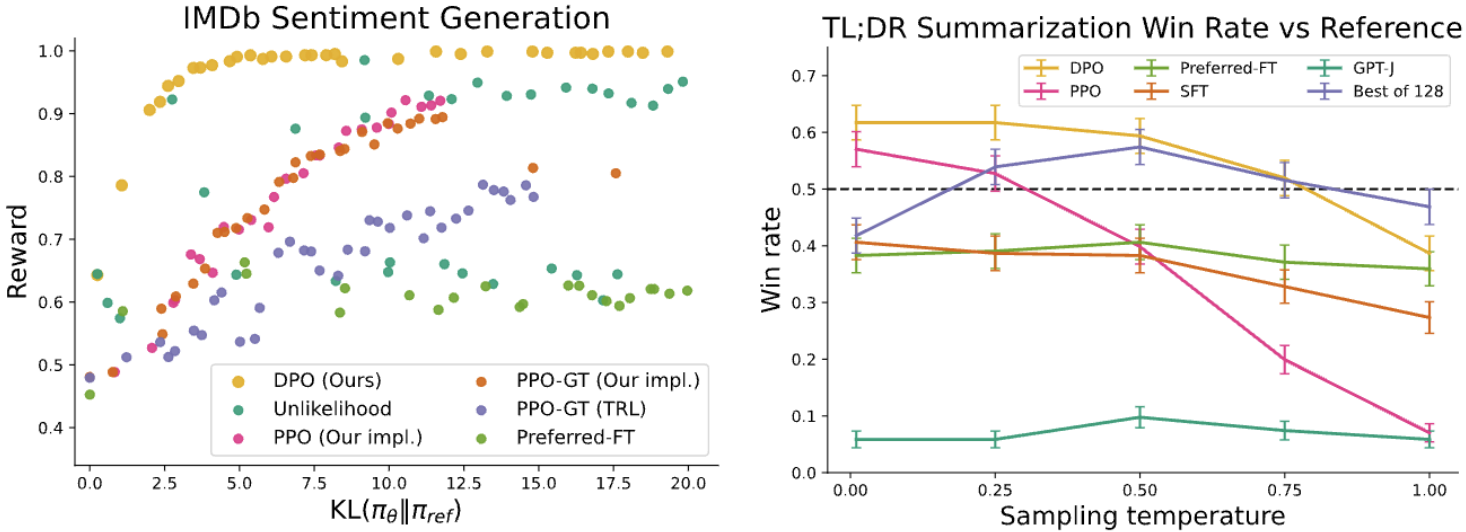


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO's best-case performance on summarization, while being more robust to changes in the sampling temperature.

PPO vs DPO who is better?

Data / Model	Alg.	Factuality	Reasoning	Coding	Truthfulness	Safety	Inst. Foll.	Average
Llama 2 base	-	52.0	37.0	30.7	32.7	32.7	-	-
TÜLU 2 (SFT)	-	55.4	47.8	45.1	56.6	91.8	44.2	56.8
StackExchange	DPO	**55.3**	**47.8**	42.4	**56.2**	92.0	46.7	56.7
	PPO	55.1	**47.8**	**46.4**	54.2	**92.6**	**47.4**	**57.3**
ChatArena (2023)	DPO	**55.4**	**50.2**	45.9	**58.5**	67.3	**50.8**	54.7
	PPO	55.2	49.2	**46.4**	55.8	**79.4**	49.7	**55.9**
HH-RLHF	DPO	**55.2**	47.6	44.2	**60.0**	**93.4**	46.6	57.8
	PPO	54.9	**48.6**	**45.9**	58.0	92.8	**47.0**	**57.9**
Nectar	DPO	**55.6**	45.8	39.0	**68.1**	**93.3**	**48.4**	58.4
	PPO	55.2	**51.2**	**45.6**	60.1	92.6	47.4	**58.7**
UltraFeedback (FG)	DPO	55.3	50.9	45.9	69.3	**91.9**	52.8	61.0
	PPO	**56.0**	**52.0**	**47.7**	**71.5**	91.8	**54.4**	**62.2**
Avg. Δ b/w PPO & DPO		-0.1	+1.3	+2.9	-2.5	+2.3	+0.1	+0.7

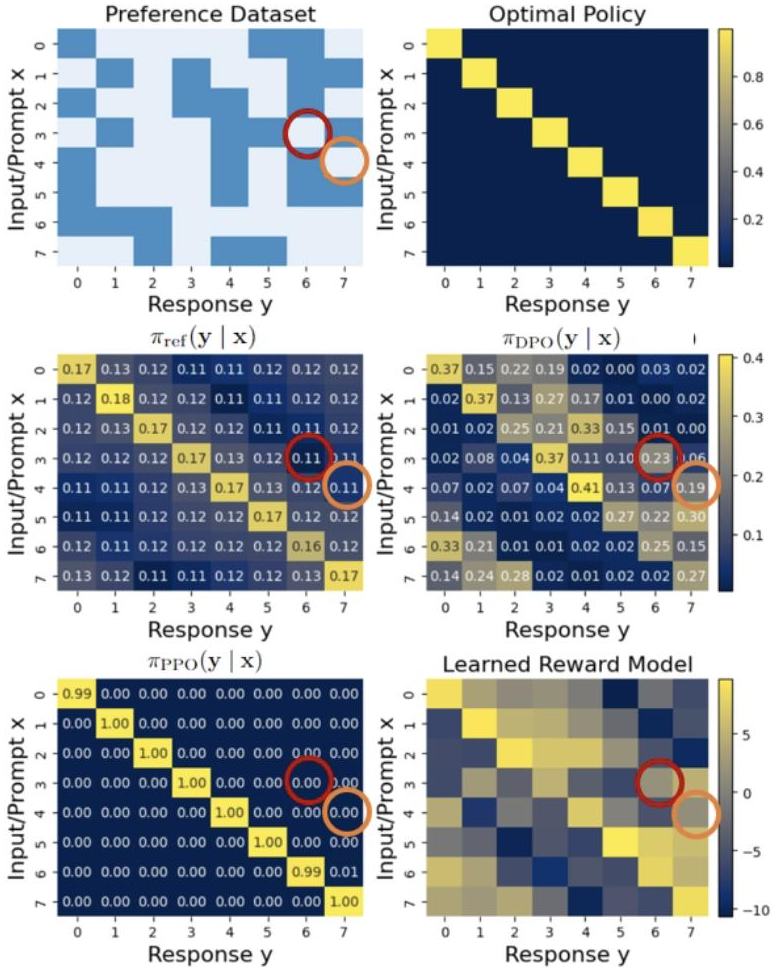
Table 2: **DPO vs PPO:** Average performance of 13B models trained using DPO and PPO across different datasets, along with the performance difference between DPO and PPO (Δ). Blue indicates improvements over the SFT baseline, orange degradations. All datasets are downsampled to 60,908 examples (except ChatArena, which is made up of 20,465 responses). PPO outperforms DPO by an average of 0.7 points, where most improvements are in reasoning, coding, and chat capabilities.

Unpacking dpo and ppo: Disentangling best practices for learning from preference feedback (2024)

PPO vs DPO who is better?

PPO is more unbiased to OOD Response Because of KL-constraint

DPO is efficient than PPO, but **susceptible** to **OOD** response generation which is unpredictable behavior



Is **dpo** superior to **ppo** for llm alignment? a comprehensive study (2024)

Odds Ratio Policy Optimization

ORPO

Limitation of instruction finetuning?

Loss is computed based on **one-hot targets.**

Fine-tuning **addresses only the correct token**.

No feedback is given for other token choices.

There are no mechanisms to penalize **rejected**

So, what's the limitation here?

$$L = -\frac{1}{m}\sum_{k=1}^{m} logP(x^k, y^k)$$

$$= -\frac{1}{m}\sum_{1}^{m}\sum_{i=1}^{|V|} y_i^k \log\left(p_i^k\right)$$

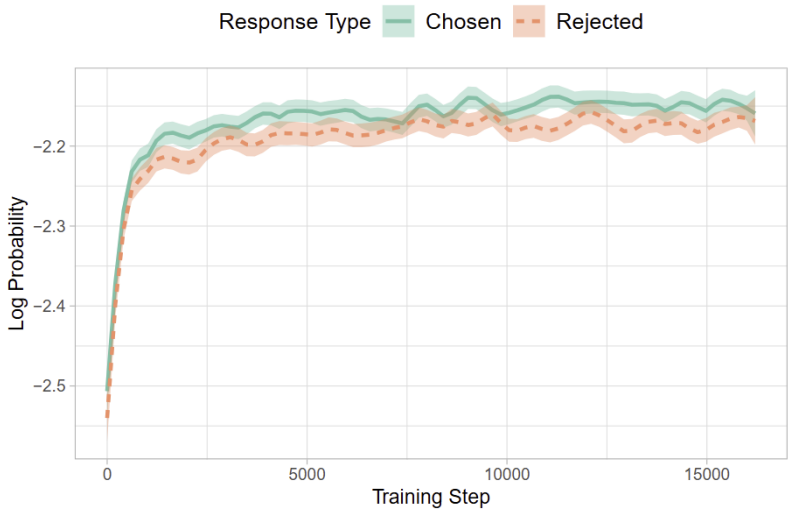
Orpo: Monolithic preference optimization without reference model

Limitation of instruction finetuning?

SFT on only chosen responses increases log-probability for both chosen and rejected responses.

Without penalizing bad outputs, rejected responses can be rated likely chosen ones.

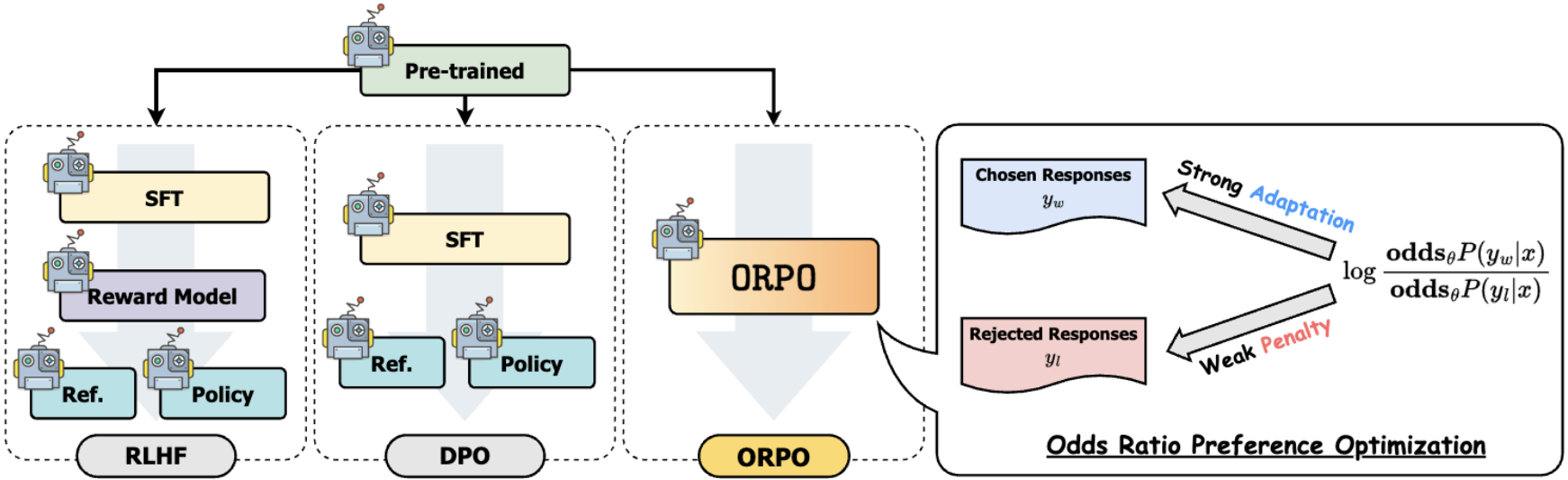
This motivates to **utilize other training method**



Orpo: Monolithic preference optimization without reference model (2024)

Odds Ratio Policy Optimization

ORPO performs RLHF as a **single-stage** process without requiring SFT



Orpo: Monolithic preference optimization without reference model (2024)

Odds Ratio Policy Optimization

$$odds_\theta(y|x) = \frac{P_\theta(y|x)}{1 - p_\theta(y|x)}$$

$odds_\theta(y|x) = k$ means the model is k times more likely to generate y than not.

$$OR_\theta(y_w, y_l) = \frac{odds_\theta(y_w|x)}{odds_\theta(y_l|x)}$$

$OR_\theta(y_w, y_l)$ shows how much more likely the model is to generate y_w over y_l given x

Orpo: Monolithic preference optimization without reference model

Odds Ratio Policy Optimization

$$L_{ORPO} = E_{(x,y_w,y_l)}[L_{SFT} + \lambda \boldsymbol{L_{OR}}] \qquad \boldsymbol{L_{OR}} = -log\delta(log\frac{odds_\theta(y_w|x)}{odds_\theta(y_l|x)})$$

Maximizes the odds of generating y_w $over$ y_l

$$\nabla_\theta L_{OR} = \delta(d) \cdot h(d)$$

$$\delta(d) = [1 + \frac{odds_\theta P(y_w|x)}{odds_\theta P(y_l|x)}]^{-1} \qquad h(d) = \frac{\nabla_\theta log P_\theta(y_w|x)}{1-P_\theta(y_w|x)} - \frac{\nabla_\theta log P_\theta(y_l|x)}{1-P_\theta(y_l|x)}$$

h(d) increases the likelihood of y_w and penalizes y_l

$\delta(d)$ becomes 0 when $P(y_w|x) \gg P(y_l|x)$
-> Prevents reward hacking!

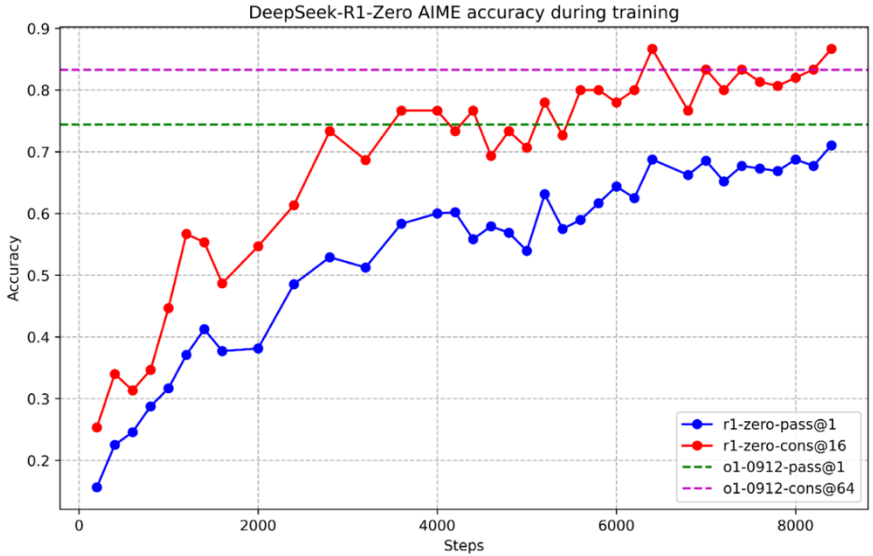
Orpo: Monolithic preference optimization without reference model

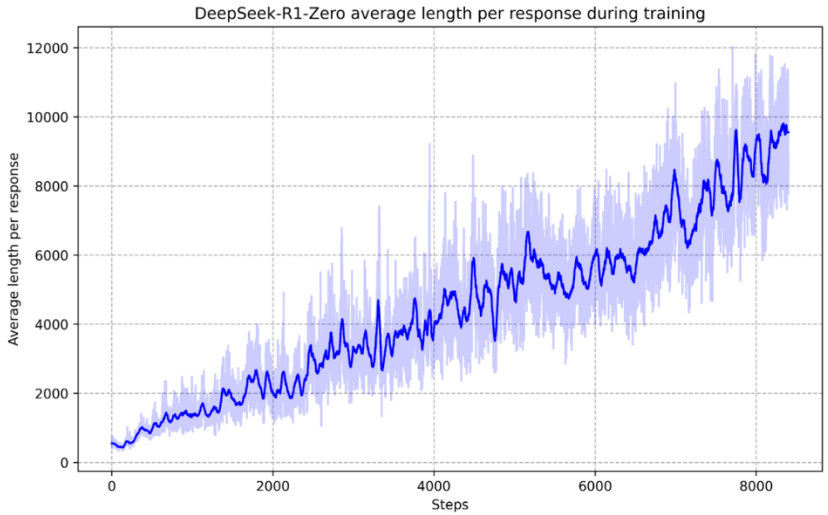
GRPO

Deepseek-R1- Zero

- **Reward modeling**
  - Accuracy reward
  - Format reward

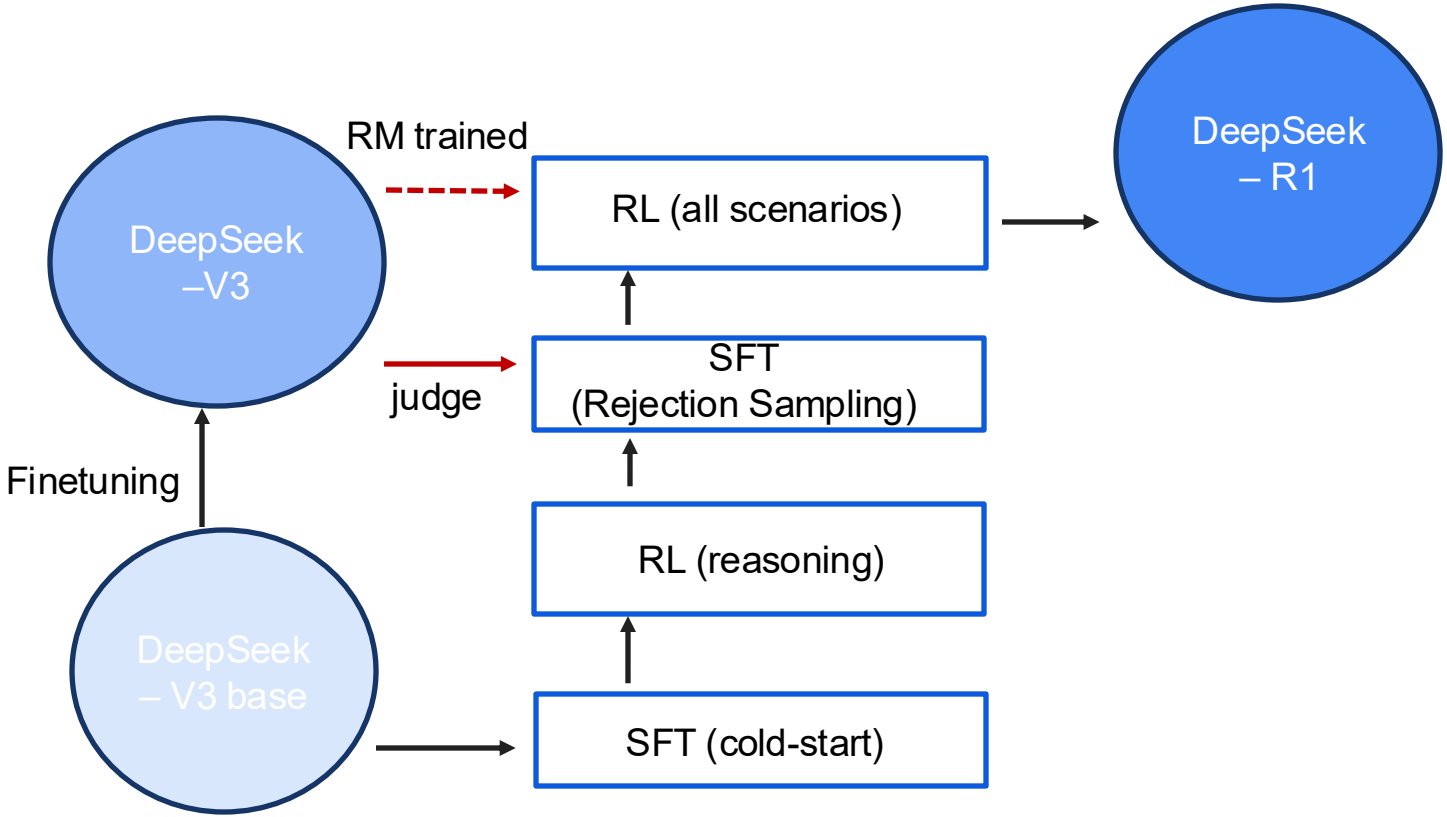
Appearance of ‘aha moment’
- reflection , exploration





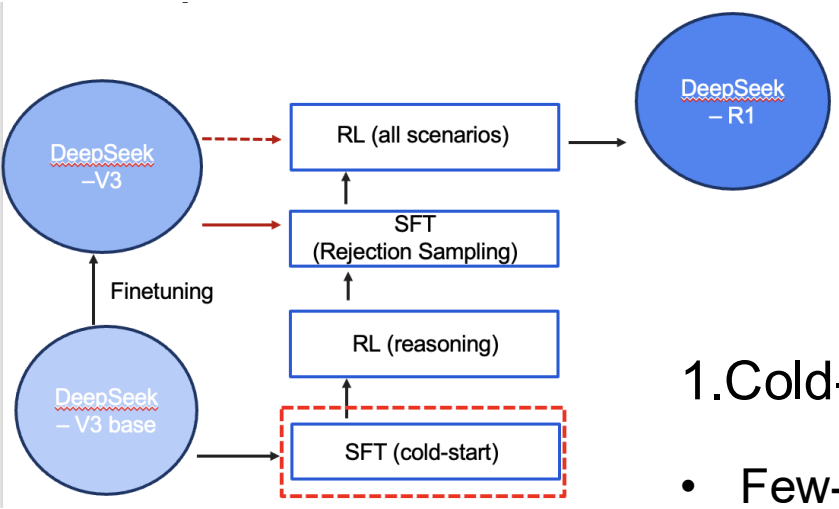
Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1



Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1

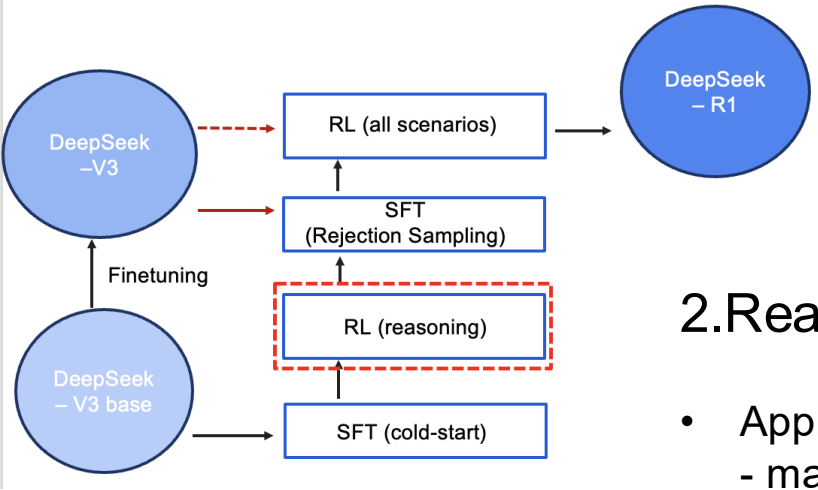


1.Cold-Start

- Few-shot prompting with long CoT examples
- Prompting to generate reflection and verification
- Refining results through annotation by human

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1

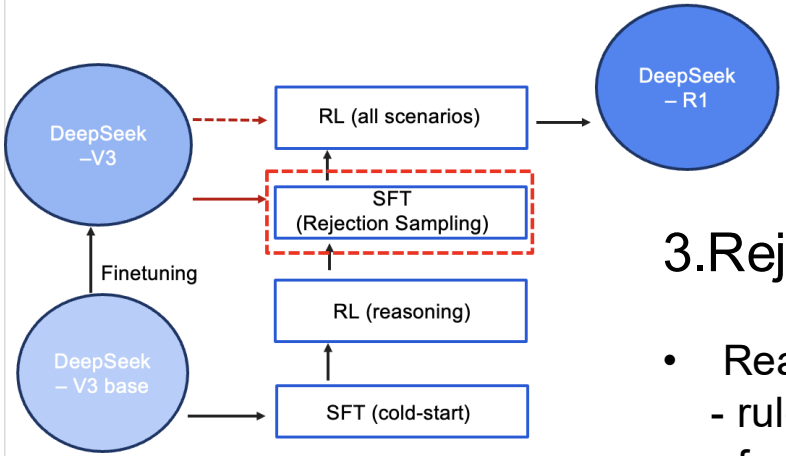


2.Reasoning Oriented RL

- Applying RL on reasoning intensive tasks
  - math , coding , science , logic …
- Reward modeling (rule based)
  - Accuracy , Format reward
  - Language consistency reward
- RL until it converges on tasks

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1



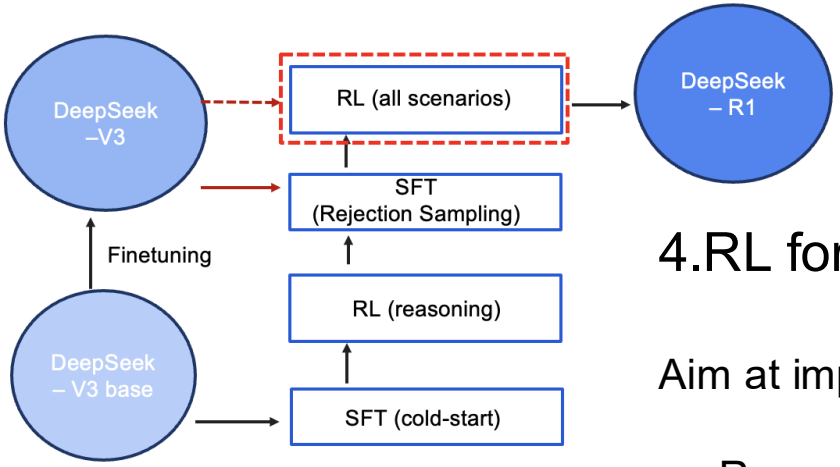
3.Rejection Sampling and SFT

- Reasoning data
  - rule based + using DeepSeek-v3 as judge
  - for each prompt, retain correct one response
  - total approximately 600k samples

- Non Reasoning data
  - reuse portions of SFT data of DeepSeek-v3
  - prompting DeepSeek-v3 to generate CoT
  - total approximately 200k samples

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1



4.RL for all scenarios

Aim at improve helpfulness & harmlessness

- Reasoning task
  - same method as Zero (rule-based)
- General task
  - Use neural RM trained on DeepSeek-V3
  - for helpfulness, focus on final summary (relevance , utility)
  - for harmlessness, evaluate entire response (reasoning + response) for mitigating biases, risks

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Deepseek-R1

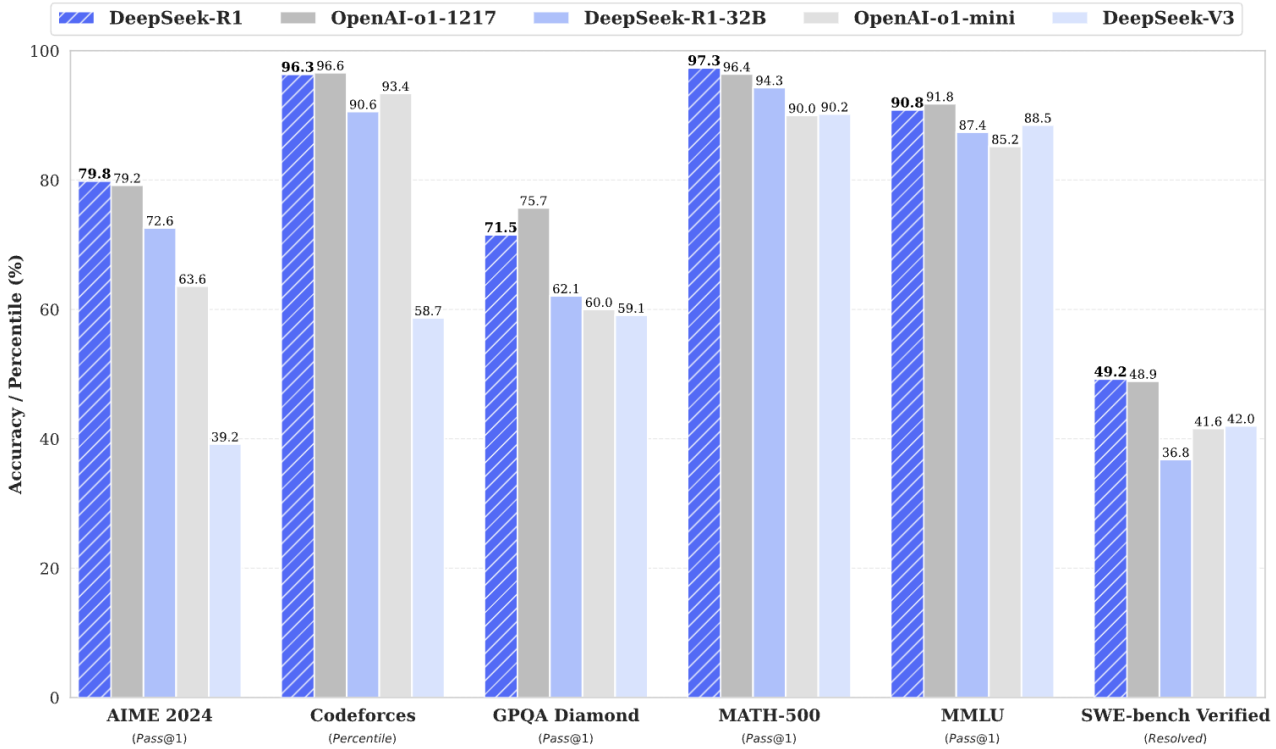


Figure 1 | Benchmark performance of DeepSeek-R1.

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)

Group Relative Policy Optimization

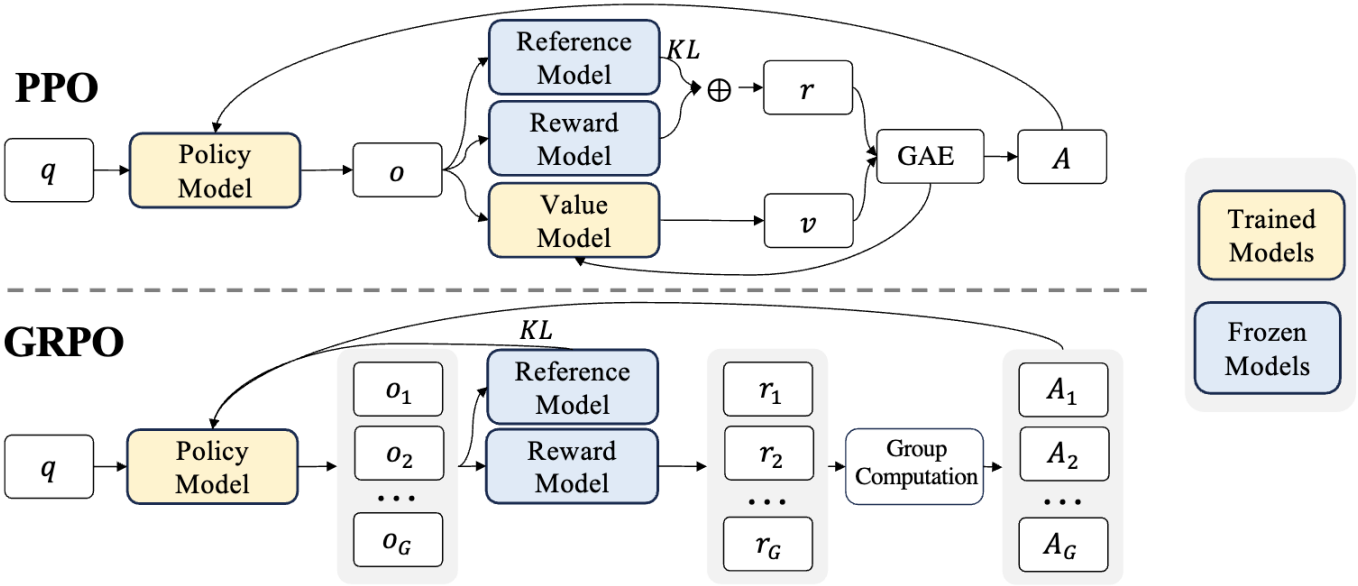


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

Deepseekmath: Pushing the limits of mathematical reasoning in open language models (2024)

Group Relative Policy Optimization

PPO (except for clipping and KL)

$$\nabla_\theta J(\pi_\theta) \approx \mathbb{E}_{\tau \sim \pi_{\theta_{old}}}\left[\sum_{t=0}^{T} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t \nabla_\theta log \pi_\theta(a_t|s_t)\right]$$

$$A_t = R_t(\tau) - V(s_t)$$

$$R_t(\tau) = \gamma^{k-t} r(s_k, a_k) \qquad V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s\right]$$

Deepseekmath: Pushing the limits of mathematical reasoning in open language models (2024)

Group Relative Policy Optimization

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right),$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1,$$

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

Difference
1.KL term (unbiased)
2.Substituting value model for reward mean

Deepseekmath: Pushing the limits of mathematical reasoning in open language models (2024)

Efficient Training

Revisit the full fine-tuning

Assume we have a pre-trained autoregressive language model $P_\phi(y|x)$

- GPT based on Transformer

Adapt this pretrained model to downstream tasks (e.g., summarization, NL2SQL, reading comprehension)

- Training dataset of context-target pairs $\{(x_i, y_i)\}_{i=1,\ldots,N}$

During full fine-tuning, we update ϕ_o to $\phi_o + \Delta\phi$ by following the gradient to maximize the conditional language modeling objective

$$\max_{\phi} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_\phi(y_t|x, y_{<t}))$$

LoRA: low rank adaptation

For each downstream task, we learn a different set of parameters $\Delta\phi$

$|\Delta\phi| = |\phi_o|$
GPT-3 has a $|\phi_o|$ of 175 billion
Expensive and challenging for storing and deploying many independent instances

Key idea: encode the task-specific parameter increment $\Delta\phi = \Delta\phi(\Theta)$ by a smaller-sized set of parameters Θ, $\Theta \ll |\phi o|$

The task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{\phi_o+\Delta\phi(\Theta)}(y_t|x, y_{<t}))$$

Lora: Low-rank adaptation of large language models. (2021)

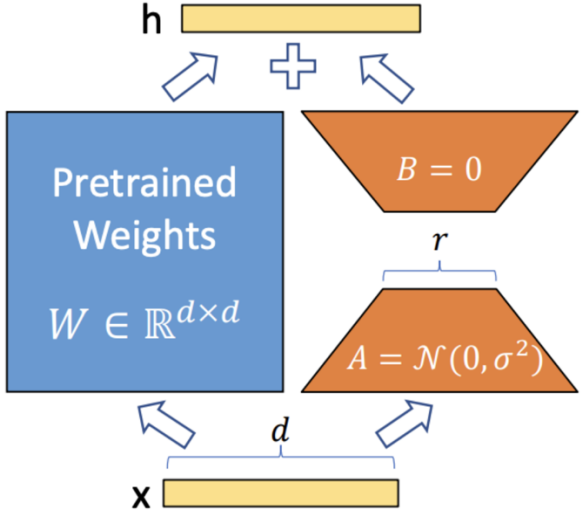
Low-rank-parameterized update matrices

$W_0 \in \mathbb{R}^{d \times k}$: a pretrained weight matrix

Constrain its update with a low-rank decomposition:
$W_0 + \Delta W = W_0 + \alpha BA$
where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

α is the tradeoff between pre-trained "knowledge" and task-specific "knowledge"

Only A and B contain **trainable** parameters



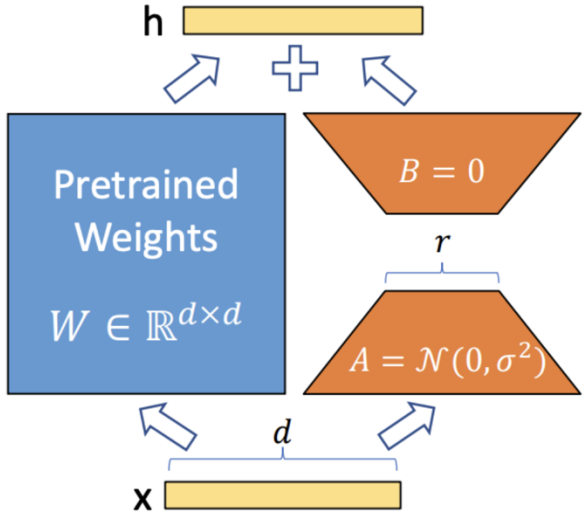
Lora: Low-rank adaptation of large language models. (2021)

Low-rank-parameterized update matrices

As one increase the number of trainable parameters, training LoRA converges to training the original model

No additional inference latency: when switching to a different task, recover $W0$ by subtracting BA and adding a different $B'A'$

Often LoRA is applied to the weight matrices in the self-attention module



Lora: Low-rank adaptation of large language models. (2021)

Example implementation of LoRA

```
input_dim = 768  # e.g., the hidden size of the pre-trained model
output_dim = 768  # e.g., the output size of the layer
rank = 8  # The rank 'r' for the low-rank adaptation

W = ... # from pretrained network with shape input_dim x output_dim

W_A = nn.Parameter(torch.empty(input_dim, rank)) # LoRA weight A
W_B = nn.Parameter(torch.empty(rank, output_dim)) # LoRA weight B

# Initialization of LoRA weights
nn.init.kaiming_uniform_(W_A, a=math.sqrt(5))
nn.init.zeros_(W_B)

def regular_forward_matmul(x, W):
    h = x @ W
return h

def lora_forward_matmul(x, W, W_A, W_B):
    h = x @ W  # regular matrix multiplication
    h += x @ (W_A @ W_B)*alpha # use scaled LoRA weights
return h
```

Appendix